

TD VII: Ingeniería de Datos

Trabajo Práctico 2

Gobierno de Datos

De los Santos, Estevez Castellano, Hsieh y Jeifetz

Junio, 2026

Índice de contenidos

1	Introducción	2
2	Arquitectura Elegida y Supuestos de Diseño	3
2.1	Contexto de la Organización y Origen de los Datos	3
2.2	Flujo de Datos bajo la Arquitectura Medallion (Data Pipeline)	4
2.3	Estrategia de Persistencia Políglota	5
2.4	Control de Acceso Basado en Roles (RBAC)	6
3	Procesamiento Masivo Distribuido	7
3.1	Descripción de los Flujos Implementados	8
3.2	Casos de Uso Relevados y Código Implementado	8
4	Análisis Relacional Complejo	12
4.1	Descripción de los Flujos Implementados	12
4.2	Casos de Uso Relevados y Código Implementado	13
5	Calidad y Validación de Datos	19
5.1	Descripción del Flujo Implementado	20
5.2	Assertions de Calidad Aplicadas al Gobierno	20
5.3	Código Implementado	21
5.4	Validación y Análisis de Resultados	23
6	Persistencia Políglota y Almacenamiento Documental	26
6.1	Documentación de la Infraestructura Elegida y Credenciales	26
6.2	Diseño de la Colección Documental	27
6.3	Casos de Uso Relevados y Consultas Implementadas	28
7	Conclusión	31

1 Introducción

En la primera etapa de este proyecto, nuestro equipo diseñó el marco fundamental de Gobierno de Datos de la organización. El objetivo inicial se centró en el diseño conceptual y lógico, y la implementación transaccional de un Catálogo Maestro y Glosario de Negocios. Utilizando un Sistema de Gestión de Bases de Datos Relacionales (RDBMS) como PostgreSQL, se modelaron las entidades críticas de la empresa: la infraestructura de datos (`FUENTE_DE_DATOS`), las lógicas analíticas (`ACTIVO`), el linaje de transformación (`LINAJE_DATOS`), el control de acceso (`SUJETO`, `ROL`, `PERFIL`, `PERMISO`) y la gestión de incidentes analíticos (`PROBLEMA`). Dicho modelo relacional demostró alta robustez al garantizar consistencia fuerte, aislamiento y atomicidad (propiedades ACID) sobre el catálogo de metadatos.

Sin embargo, el diagnóstico de madurez de la organización reveló que el crecimiento operacional introduce desafíos de Big Data que exceden las capacidades de un motor relacional monolítico. El Catálogo de Gobierno de Datos no puede comportarse como un inventario estático alimentado de forma manual; sino que requiere nutrirse dinámicamente de flujos masivos de eventos, telemetría de infraestructura e ingestas de sistemas externos operando con diversas cadencias, formatos y volúmenes.

Esta segunda entrega toma el modelo relacional previamente diseñado y lo inserta en una arquitectura corporativa distribuida y polígota. El presente informe describe el diseño arquitectónico propuesto por nuestro equipo bajo el patrón Medallion, el despliegue de procesamientos masivos distribuidos utilizando Apache Spark (para auditar e interrelacionar metadatos a escala), la configuración de una barrera automatizada de calidad de datos mediante Great Expectations, y la flexibilidad del modelo documental con MongoDB.

2 Arquitectura Elegida y Supuestos de Diseño

Como responsables de la estrategia de datos de la organización, determinamos que la solución debía extender el catálogo del TP1 hacia una arquitectura distribuida orientada al procesamiento analítico. Para ello, adoptamos un enfoque basado en la Arquitectura Medallion y persistencia políglota. Esta decisión permite integrar datos provenientes de múltiples sistemas operacionales del cliente, garantizar controles de calidad automatizados y ofrecer mecanismos de almacenamiento eficientes según el rol del consumidor final.

2.1 Contexto de la Organización y Origen de los Datos

Para asegurar que el catálogo estuviera sincronizado con la realidad operativa de la empresa, integramos tres sistemas de registro heterogéneos:

Sistemas de Recursos Humanos

Contiene los datos maestros de la estructura organizacional: empleados (`MIEMBRO_UO`), puestos y departamentos.

- **Cadencia y formato:** Procesamiento por batch diario ejecutado a las 00:00 hs. Exporta archivos compactos en formato Parquet hacia el Data Lake.
- **Volumen:** Bajo a medio (cambios diarios incrementales).
- **Decisión de Gobierno:** Para mantener la trazabilidad en el catálogo, el pipeline gestiona Slowly Changing Dimensions (SCD). Ante un cambio de rol, se cierra el registro histórico (actualizando `fecha_salida`) y se abre una nueva tupla, preservando el linaje de accesos históricos.

Sistema de Gestión de Servicios IT

Registra la gobernanza operativa del día a día: tickets de incidentes (`PROBLEMA`), y solicitudes de acceso.

- **Cadencia y formato:** Procesamiento en micro-batch cada 15-30 minutos. Los datos arriban como documentos JSON anidados.
- **Volumen:** Medio (miles de registros semanales).
- **Decisión de Gobierno:** Dado que las APIs imponen límites de tasa, el pipeline implementa paginación avanzada y un patrón de reintentos con Exponential Backoff.

Logs de Infraestructura y Auditoría de Accesos

Captura en tiempo real qué usuario ejecutó qué consulta sobre qué base de datos física (ACCESO_FUENTE_DATOS).

- **Cadencia y formato:** Flujo continuo (Streaming) mediante colas de mensajería distribuidas hacia archivos de texto plano.
- **Volumen:** Big Data (terabytes de logs diarios).
- **Decisión de Gobierno:** En un escenario de producción real, la velocidad y el volumen de este flujo imposibilitarían la escritura directa en PostgreSQL sin degradar el rendimiento transaccional del catálogo, razón por la cual se diseñó una landing zone dedicada y un motor analítico como Spark para preprocesar las métricas de auditoría.

2.2 Flujo de Datos bajo la Arquitectura Medallion (Data Pipeline)

Diseñamos el ciclo de vida del dato bajo un enfoque ELT (Extract, Load, Transform), estructurando el Data Lake en tres capas de madurez:

- **Capa Bronze (Ingesta Cruda):** Los datos aterrizan inalterados. Siguen un patrón Append-Only inmutable, proveyendo la "fuente de la verdad" ante necesidades de auditoría legal (Compliance).

- **Capa Silver (Validación):** Un proceso aplica estandarizaciones y un control de calidad severo (Data Quality Gate) mediante Great Expectations. Los registros inválidos se desvían a una Dead Letter Queue, protegiendo el catálogo de negocio de contaminación externa.
- **Capa Gold (Consumo):** Mediante transformaciones en Apache Spark, se consolidan las lógicas de negocio (ej. agregación de accesos). Desde aquí, los datos refinados alimentan la persistencia políglota para el consumo final.

Se optó por un enfoque ELT en lugar de ETL tradicional dado que, en un escenario de producción real, este patrón permite reducir la latencia de ingesta al postergar las transformaciones hacia la capa Gold, donde Spark puede operar sobre los datos ya almacenados en el lago sin necesidad de moverlos a un motor externo.

2.3 Estrategia de Persistencia Políglota

Para resolver las limitaciones lógicas y físicas del RDBMS transaccional clásico, la capa de datos se bifurca según los casos de uso:

- **PostgreSQL (Estructurado / Relacional):** Retiene el Catálogo Maestro y Glosario de Negocios implementados en el TP1. Almacena las jerarquías organizacionales, estructuras de activos y matrices de roles que exigen consistencia ACID estricta e integridad referencial fuerte, operando con volúmenes moderados de mutación.
- **MongoDB (NoSQL Documental / Semi-estructurado):** Absorbe los historiales documentales de los tickets dinámicos de servicios IT (los cuales mutan sus atributos con comentarios adjuntos o etiquetas variables) y los reportes masivos de auditoría generados por Great Expectations. Ofrece un esquema flexible y escalabilidad horizontal óptima para almacenar documentos JSON de auditoría sin penalizar el rendimiento relacional.

2.4 Control de Acceso Basado en Roles (RBAC)

El modelo de seguridad perimetral de la arquitectura restringe el acceso a los componentes lógicos según el principio de menor privilegio:

- **Data Governance Lead:** Rol estratégico. Define las expectativas de calidad en código mediante Great Expectations y audita reportes agregados en MongoDB. Posee permisos de solo lectura global.
- **Data Steward:** Administrador funcional del catálogo. Atiende incidentes y actualiza términos directamente mediante accesos de escritura transaccional en PostgreSQL.
- **Data Engineer:** Responsable del desarrollo de flujos en Spark y de la codificación de reglas de calidad. Posee accesos de lectura y escritura sobre las capas Bronze y Silver, clusters de Spark y colecciones de depuración en MongoDB.
- **Database Administrator (DBA):** Gestiona copias de seguridad, consistencia de réplicas en MongoDB y optimización física de índices en PostgreSQL. Posee permisos a nivel de infraestructura, sin acceso lógico al dato de negocio.
- **Data Analyst:** Accede a vistas optimizadas de solo lectura en PostgreSQL para comprender el glosario semántico y construir reportes de inteligencia de negocios.
- **Data Scientist:** Consume archivos del Data Lake y documentos almacenados en MongoDB para desarrollar modelos analíticos y predictivos.
- **Service Accounts:** Cuentas de automatización no humanas (por ejemplo, orquestadores de tareas). Operan con credenciales aisladas y los permisos mínimos necesarios para leer de una capa del Data Lake y escribir en la siguiente.
- **Chief Information Security Officer (CISO):** Rol estratégico y de cumplimiento. Audita las políticas de acceso, evalúa vulnerabilidades en la infraestructura y vela por

el cumplimiento normativo. Posee permisos de solo lectura global y acceso irrestricto a los reportes de auditoría de accesos generados en la capa Gold mediante Spark.

Para validar la arquitectura propuesta en los siguientes apartados, el presente trabajo se desarrolló como una simulación en un entorno controlado. Los datos utilizados para alimentar las herramientas de procesamiento y validación (Spark, Great Expectations y MongoDB) provienen directamente de los registros generados mediante el DML (`dml_gobierno_datos.sql`) de la primera entrega (PostgreSQL). Con fines de demostración técnica, el alcance se restringió a un recorte específico del dominio, enfocado principalmente en la gestión de incidentes, infraestructura y accesos, empleando un volumen representativo de registros. Esto permite probar la viabilidad técnica de la persistencia políglota y el procesamiento distribuido en un caso de uso acotado, demostrando las capacidades de las herramientas sin la sobrecarga operativa de un entorno de producción a gran escala. Las justificaciones de negocio presentadas a continuación asumen el rol de un equipo que diseña la gobernanza corporativa sobre este entorno simulado.

Cabe destacar que la totalidad de las consultas analíticas, scripts y pipelines desarrollados para cada una de las secciones se encuentran completamente implementados y disponibles en archivos de Google Colab, listos para ser corridos, ejecutados y verificados de manera directa.

3 Procesamiento Masivo Distribuido

Definida la arquitectura, el primer desafío fue auditar la infraestructura del cliente. Para ello, se implementó un motor de procesamiento distribuido basado en Resilient Distributed Datasets (RDDs) mediante Apache Spark, sustentado en el paradigma MapReduce. Este enfoque permite procesar grandes volúmenes de metadatos en paralelo sin saturar los sistemas operacionales, habilitando auditorías automatizadas sobre el catálogo de gobierno.

3.1 Descripción de los Flujos Implementados

El flujo opera bajo evaluación perezosa:

1. **Carga (Context):** Los datos extraídos del Data Lake se particionan en el clúster, generando los RDDs (`parallelize()`).
2. **Filtrado y Transformación:** Se limpia el dominio (`filter()`) y la operación `map()` tokeniza las cadenas emitiendo tuplas $(K, 1)$.
3. **Agregación Distribuida:** `reduceByKey()` suma las frecuencias asociadas a cada clave $(a + b)$ mediante *combiners* locales y globales.
4. **Exposición:** `collect()` devuelve el informe al nodo coordinador.

3.2 Casos de Uso Relevados y Código Implementado

A continuación, se describen los seis escenarios de auditoría implementados para el cliente:

1. Distribución del Capital Humano (SUJETO)

Las políticas corporativas exigen controlar la proporción de usuarios externos vs. internos. Esta métrica audita masivamente las entidades en el sistema para detectar desviaciones de acceso.

```
1 map_sujetos = sujetos_rdd.map(lambda x: (x.split(',')[1], 1))
2 reduce_sujetos = map_sujetos.reduceByKey(lambda a, b: a + b)
3 print(reduce_sujetos.collect())
```

Resultado obtenido:

```
[('Miembro UO', 3), ('Externo', 2), ('Organizacion', 2)]
```

Análisis: La plataforma cuenta con 3 usuarios internos. Sin embargo, la suma de entidades externas y organizaciones (4) supera a los miembros internos, justificando la necesidad de aplicar, en un entorno real, políticas de auditoría perimetral estrictas.

2. Madurez del Catálogo por Fase de Vida (ACTIVO)

Evalúa la salud operativa del ciclo de vida de los activos. Permite al Data Governance Lead detectar cuellos de botella organizacionales (ej. activos estancados en fase de "Creación").

```
1 map_activos = activos_rdd.map(lambda x: (x.split(',')[3], 1))
2 reduce_activos = map_activos.reduceByKey(lambda a, b: a + b)
3 print(reduce_activos.collect())
```

Resultado obtenido:

```
[('3', 3), ('2', 3)],
```

donde '2' = Almacenamiento y '3' = Uso siguiendo el catálogo de fases del TP1.

Análisis: El inventario simulado se encuentra balanceado equitativamente entre las fases 2 (Almacenamiento) y 3 (Uso). Esto indica un ciclo de desarrollo saludable donde los activos logran superar las etapas tempranas y llegan a ser consumidos.

3. Diversificación de la Arquitectura Tecnológica (HERRAMIENTA)

Provee a la gerencia un inventario automatizado (Software propietario vs. Open Source). Estratégicamente vital para mitigar el riesgo de dependencia tecnológica y controlar costos de licenciamiento.

```
1 map_herramientas = herramientas_rdd.map(lambda x: (x.split(',')[2], 1))
2 reduce_herramientas = map_herramientas.reduceByKey(lambda a, b: a + b)
3 print(reduce_herramientas.collect())
```

Resultado obtenido:

```
[('Software', 4), ('Licencia', 1), ('Herramienta Open Source', 1)]
```

Análisis: Existe una clara dependencia funcional sobre el Software propietario y Licencias (5 en total), frente a una adopción mínima de herramientas Open Source (1). Esto representa, en entorno real, una oportunidad gerencial para reestructurar costos IT en futuras migraciones.

4. Balance de Calidad de Datos Operativa (PROBLEMA)

Proporciona un KPI inmediato sobre la eficiencia del equipo de Data Stewards, contrastando el volumen de incidentes resueltos frente a la deuda técnica de la organización.

```
1 map_problemas = problemas_rdd.map(lambda x: (x.split(',')[3], 1))
2 reduce_problemas = map_problemas.reduceByKey(lambda a, b: a + b)
3 print(reduce_problemas.collect())
```

Resultado obtenido:

```
[('Resuelto', 1), ('No Resuelto', 3)]
```

Análisis: La deuda técnica (No Resuelto) triplica a los incidentes solucionados en la muestra actual. Este KPI, en entorno real, evidenciaría que el equipo de soporte se encuentra saturado o que existen bloqueantes técnicos severos que demoran la resolución.

5. Auditoría de Permisos por Perfil (ACCESO_FUENTE_DATOS)

Facilita al CISO verificar el principio de menor privilegio, alertando rápidamente sobre concentraciones anómalas de autorizaciones críticas.

```
1 map_accesos = accesos_rdd.map(lambda x: (x.split(',')[3], 1))
2 reduce_accesos = map_accesos.reduceByKey(lambda a, b: a + b)
3 print(reduce_accesos.collect())
```

Resultado obtenido:

```
[('4', 4), ('1', 1), ('2', 1), ('5', 1)],
```

donde los valores corresponden a los perfiles definidos en el catálogo: '1' = Lectura Total, '2' = Lectura Restringida, '3' = Escritura, '4' = Administración, '5' = Auditoría.

Análisis: Existe una sobre-asignación de accesos correspondientes al Perfil 4 (Alta jerarquía/Administrador según el catálogo). Esto en un entorno real, representaría un riesgo perimetral que el Oficial de Seguridad (CISO) debe auditar y revocar inmediatamente.

6. Monitoreo de Vulnerabilidades Críticas (PROBLEMA)

Aísla inteligentemente los problemas activos para identificar qué bases de datos físicas son el punto de falla recurrente, permitiendo al DBA priorizar sus esfuerzos de remediación.

```
1 map_fuentes_criticas = problemas_rdd.filter(  
2     lambda x: x.split(',')[3] == 'No Resuelto'  
3 ).map(lambda x: (x.split(',')[4], 1))  
4 reduce_fuentes_criticas = map_fuentes_criticas.reduceByKey(lambda a, b: a  
5     + b)  
5 print(reduce_fuentes_criticas.collect())
```

Resultado obtenido:

```
[('3', 1), ('4', 1), ('1', 1)]
```

Análisis: La reducción aisló la deuda técnica y determinó que las fuentes físicas '1', '3' y '4' requieren mantenimiento prioritario. Esto en un entorno real provee un diagnóstico directo y exacto para que el equipo de Infraestructura focalice sus esfuerzos de remediación.

La ejecución de estos seis procesos de bajo nivel se realizó sobre un recorte controlado del DML original, con el objetivo de validar la eficacia del paradigma MapReduce para la auditoría de metadatos. Al operar sobre esta muestra simulada, comprobamos empíricamente que el motor es capaz de aislar indicadores críticos de gobierno de manera exitosa.

A pesar de trabajar con un volumen reducido de registros, la prueba de concepto arrojó insights precisos que validan la utilidad de la herramienta:

1. **Detección de Deuda Técnica:** El sistema logró identificar y aislar correctamente la concentración de incidentes "No Resueltos", mapeándolos a sus fuentes físicas específicas, demostrando cómo se podría guiar el esfuerzo de mantenimiento de un DBA en un entorno real.
2. **Auditoría de Accesos Perimetrales:** La agregación distribuida detectó con éxito la proporción de perfiles de alta jerarquía (Administradores) dentro del recorte de datos,

validando que el script es funcional para ejecutar auditorías del principio de menor privilegio.

En conclusión, esta simulación demuestra que la lógica distribuida implementada es correcta. Al escalar este modelo a la volumetría real (Big Data) de la capa Bronze del Data Lake, la organización contará con una herramienta ágil para auditar su infraestructura sin sobrecargar sus bases operacionales.

4 Análisis Relacional Complejo

Mientras MapReduce nos permitió auditar la salud general de la infraestructura cruda, las preguntas de negocio de la gerencia exigen interrelacionar entidades. Para satisfacer la necesidad de rastrear linajes semánticos e impacto operativo en la capa Gold, instanciamos el modelo relacional del catálogo en DataFrames de Spark SQL.

4.1 Descripción de los Flujos Implementados

El flujo analítico desarrollado en Spark SQL se diseñó como un pipeline secuencial en memoria para interactuar con los datos de la simulación mediante consultas declarativas estándar. El proceso se estructuró en tres pasos operativos:

1. **Carga y Tipado de DataFrames:** Los registros provenientes del recorte del DML se cargaron en memoria estructurándose como DataFrames de Spark. En esta fase se les aplicó un esquema tipado que replica las restricciones de tablas del modelo relacional original del TP1 (claves y tipos de datos), preparando los datos para el análisis.
2. **Creación de Vistas Virtuales:** Para poder aplicar sintaxis SQL sobre los DataFrames, cada uno se registró en el catálogo en memoria mediante `createOrReplaceTempView`. Esto genera vistas temporales que permiten consultar las tablas simuladas directamente, sin necesidad de duplicar los datos ni realizar escrituras en disco.

3. **Ejecución Lógica y Exposición:** Al invocar el motor mediante `spark.sql()`, las consultas se procesan a través del optimizador nativo de Spark, el cual determina la manera más rápida de resolver los joins y filtros aplicados. Finalmente, el flujo se materializa en consola mediante la acción `.show()`, devolviendo las tablas de control finales que cruzan la infraestructura técnica con las métricas de negocio.

4.2 Casos de Uso Relevados y Código Implementado

1. Trazabilidad e Impacto Técnico

Ante la caída de una fuente física, el sistema traza automáticamente qué activos lógicos se verán interrumpidos y qué equipos deben ser notificados, habilitando una respuesta proactiva antes de que el impacto llegue a la toma de decisiones.

```

1 query_1 = '''
2     SELECT f.nombre AS Fuente_Origen,
3           a.nombre AS Objeto,
4           e.nombre AS Equipo_Responsable
5     FROM fuente_de_datos f
6     JOIN fuente_impacta_objeto fio ON f.id_fuente = fio.id_fuente
7     JOIN activo a ON fio.id_activo = a.id_activo
8     JOIN equipo_trabaja_objeto eto ON a.id_activo = eto.id_activo
9     JOIN equipo e ON eto.id_equipo = e.id_equipo
10  '''
11 resultado_1 = spark.sql(query_1)
12 resultado_1.show(truncate=False)

```

Listing 1: Consulta de Causa Raíz e Impacto Operativo

Resultado obtenido:

Fuente	Objeto	Equipo
DWH Finanzas Oracle	Tablero Retencion y Churn	Contabilidad General

```
|DWH Finanzas Oracle |Reporte Ingreso Mensual |Contabilidad General |
|DWH Ventas SQL Server|ETL Ingesta Ventas          |Datos e Inteligencia |
+-----+-----+
```

Análisis: Logramos mapear la cadena de impacto completa. Por ejemplo, ante una caída del DWH Finanzas Oracle, sabemos automáticamente que el área de Contabilidad General perderá acceso a su Reporte de Ingreso y Tablero de Churn, permitiendo, en un entorno real, una comunicación proactiva de incidentes.

2. Impacto de Incidentes en Operaciones de Negocio

Traduce alertas técnicas al lenguaje ejecutivo: identifica qué operaciones corporativas críticas están en riesgo de frenarse por incidentes de datos abiertos.

```
1 query_2 = '''
2     SELECT p.descripcion AS Falla_Reportada,
3           a.nombre AS Activo_Afectado,
4           o.descripcion AS Operacion_Impactada
5     FROM problema p
6     JOIN activo a ON p.id_activo = a.id_activo
7     JOIN operacion_vinculada_activo ova ON a.id_activo = ova.id_activo
8     JOIN operacion o ON ova.id_operacion = o.id_operacion
9     WHERE p.estado = 'No Resuelto'
10  '''
11 resultado_2 = spark.sql(query_2)
12 resultado_2.show(truncate=False)
```

Listing 2: Mapeo de Incidentes contra Procesos Operativos

Resultado obtenido:

```
+-----+-----+-----+
|Falla_Reportada|Activo_Afectado|Operacion_Impactada|
+-----+-----+-----+
+-----+-----+-----+
```

Análisis: El motor arrojó un conjunto vacío. Lejos de ser un error, este es un insight sumamente positivo: demuestra que ninguna de las fallas técnicas actuales está bloqueando Operaciones Críticas de Negocio en este instante, manteniendo el riesgo operativo aislado.

3. Data Lineage y Trazabilidad Regulatoria

Reconstruye el árbol genealógico del dato, permitiendo a auditores verificar la proveniencia de cualquier tablero o modelo predictivo para cumplimiento normativo.

```

1 query_3 = '''
2     SELECT origen.nombre AS Proceso_Padre,
3           destino.nombre AS Objeto_Hijo,
4           destino.tipo_activo
5     FROM linaje_datos l
6     JOIN activo origen ON l.id_activo_origen = origen.id_activo
7     JOIN activo destino ON l.id_activo_destino = destino.id_activo
8 '''
9 resultado_3 = spark.sql(query_3)
10 resultado_3.show(truncate=False)

```

Listing 3: Grafo de Linaje de Datos

Resultado obtenido:

Proceso_Padre	Objeto_Hijo	tipo_activo
Tabla Maestra Clientes	Reporte Ingreso Mensual	Objeto
Tabla Maestra Clientes	Tablero Retencion y Churn	Objeto
ETL Ingesta Ventas	Tabla Maestra Clientes	Datos Estructurado
Tabla Maestra Clientes	Modelo Predictivo Churn	Modelo

Análisis: Identificamos que la Tabla Maestra Clientes es un nodo crítico del linaje, ya que su alteración impactaría simultáneamente en capas de visualización (Reportes y Tableros)

y en algoritmos de Machine Learning (Modelo Predictivo Churn). En un entorno real, este nodo requeriría controles de cambio estrictos.

4. Soporte Tecnológico de Infraestructura vs. Activos

Genera un inventario vivo del stack tecnológico, identificando qué herramientas y licencias vigentes sustentan qué activos lógicos, y detectando tecnologías sobredemandadas o licencias subutilizadas candidatas a ser dadas de baja.

```

1 query_4 = '''
2     SELECT h.nombre AS Herramienta_Proveedor,
3           a.nombre AS Activo_Implementado,
4           a.tipo_activo
5     FROM herramienta h
6     JOIN herramienta_gestiona_activo hga ON h.id_herramienta = hga.
       id_herramienta
7     JOIN activo a ON hga.id_activo = a.id_activo
8 '''
9 resultado_4 = spark.sql(query_4)
10 resultado_4.show(truncate=False)

```

Listing 4: Mapeo de Dependencias de Infraestructura Tecnológica

Resultado obtenido:

Herramienta_Proveedor	Activo_Implementado	tipo_activo
Power BI	Reporte Ingreso Mensual	Objeto
Databricks Lakehouse	Modelo Predictivo Churn	Modelo
AWS Glue	ETL Ingesta Ventas	Objeto

Análisis: Conseguimos un inventario vivo de licenciamiento. Validamos la integración de tecnologías Cloud (AWS Glue) para la ingesta, una arquitectura de datos basada en el modelo

Medallion para el almacenamiento y transformación de la información, y software propietario (Power BI) para el consumo estructurado, permitiendo detectar licencias subutilizadas o sobredemandadas.

5. Auditoría de Seguridad Perimetral y Accesos Vigentes

Automatiza la matriz de accesos perimetrales, permitiendo al CISO verificar qué sujetos están autorizados sobre cada fuente física y validar que sus privilegios sean consistentes con sus responsabilidades corporativas.

```

1 query_5 = '''
2     SELECT s.id_sujeto,
3            s.tipo_sujeto,
4            f.nombre AS Fuente_Autorizada,
5            afd.id_perfil AS Nivel_Privilegio
6     FROM sujeto s
7     JOIN acceso_fuente_datos afd ON s.id_sujeto = afd.id_sujeto
8     JOIN fuente_de_datos f ON afd.id_fuente = f.id_fuente
9     ORDER BY s.id_sujeto
10  '''
11 resultado_5 = spark.sql(query_5)
12 resultado_5.show(truncate=False)

```

Listing 5: Auditoría y Control de Seguridad Perimetral de Accesos

Resultado obtenido:

id_sujeto	tipo_sujeto	Fuente_Autorizada	Nivel_Privilegio
1001	Miembro UO	DWH Finanzas Oracle	4
1001	Miembro UO	DWH Ventas SQL Server	4
1010	Miembro UO	DWH Finanzas Oracle	2

Análisis: La matriz perimetral alerta al CISO sobre el Sujeto 1001, quien concentra privile-

gios de Nivel 4 (Administración) sobre múltiples bases de datos críticas simultáneamente. En un entorno real, este acceso debe ser revalidado periódicamente para garantizar el principio de menor privilegio.

6. Medición de Carga de Trabajo Analítica por Equipo

Cuantifica el volumen de activos asignados por equipo, detectando desbalances de responsabilidad que aporten justificación basada en datos para redistribuir tareas o planificar contrataciones estratégicas.

```

1 query_6 = '''
2     SELECT e.nombre AS Equipo_Gestor,
3           COUNT(a.id_activo) AS Volumen_Objetos_Asignados
4     FROM equipo e
5     JOIN equipo_trabaja_objeto eto ON e.id_equipo = eto.id_equipo
6     JOIN activo a ON eto.id_activo = a.id_activo
7     GROUP BY e.nombre
8 '''
9 resultado_6 = spark.sql(query_6)
10 resultado_6.show(truncate=False)

```

Listing 6: Cálculo de KPI de Carga de Trabajo por Equipo Técnico

Resultado obtenido:

```

+-----+-----+
|Equipo_Gestor      |Volumen_Objetos_Asignados|
+-----+-----+
|Contabilidad General |2                        |
|Analitica de Clientes|2                        |
|Datos e Inteligencia |1                        |
+-----+-----+

```

Análisis: Detectamos un desbalance en la asignación de responsabilidades. Los equipos de negocio (Contabilidad y Analítica) mantienen el doble de activos que el equipo central de

Datos e Inteligencia, indicando, en un escenario de escala real, la necesidad de redistribuir la propiedad de los activos mediante una política formal de Data Ownership.

Al igual que en MapReduce, operamos sobre un recorte del DML para validar la capacidad de Spark SQL en la resolución de interrelaciones semánticas complejas. El objetivo fue comprobar empíricamente que el motor distribuido puede responder preguntas de negocio de alto costo computacional sin saturar sistemas transaccionales.

Trabajando sobre esta muestra, la simulación arrojó insights precisos que validan el diseño de las consultas:

1. **Trazabilidad de Impacto Organizacional:** El motor logró reconstruir con exactitud la cadena de dependencia desde las fuentes físicas (ej. "DWH Finanzas Oracle") hasta los activos lógicos y los equipos responsables (ej. "Contabilidad General"). Esto demuestra que la arquitectura puede automatizar alertas tempranas y mitigar el impacto operativo ante caídas de servidores.
2. **Linaje de Datos (*Compliance*):** El Autocruce (*Self-Join*) jerárquico mapeó correctamente las dependencias funcionales entre las tablas maestras y los activos de consumo final, validando su aptitud para auditorías normativas.

En síntesis, la obtención exitosa de estas métricas transversales demuestra que la arquitectura lógica es plenamente funcional. Al escalar hacia la volumetría corporativa real, Spark SQL proporcionará a la gerencia la visibilidad exacta y necesaria para gobernar el ecosistema de datos.

5 Calidad y Validación de Datos

Habiendo demostrado el poder analítico de Spark en nuestra arquitectura, detectamos un riesgo: las conclusiones analíticas son inútiles si la materia prima está corrupta. Dado que el

Data Lake en la capa Bronze no posee las validaciones ACID que sí teníamos en PostgreSQL en el TP1, diseñamos un Data Quality Gate transversal. Utilizando el framework **Great Expectations**, implementamos un peaje programático entre la capa Bronze y Silver para garantizar las políticas de gobierno del cliente.

5.1 Descripción del Flujo Implementado

El pipeline de validación se estructuró en una función centralizada de Python que evalúa los datos siguiendo estos cuatro pasos lógicos:

1. **Carga en Memoria:** Se inicializa un entorno temporal del framework y se le inyecta el DataFrame con los datos de la capa Bronze para su análisis.
2. **Definición de Reglas:** Se construye una "Suite de Expectativas", que funciona como un contrato de datos que agrupa las reglas de negocio que cada registro debe cumplir obligatoriamente.
3. **Ejecución y Diagnóstico:** El motor evalúa cada fila contra el contrato. En lugar de simplemente alertar que algo falló, se configuró el sistema para que extraiga de forma exacta la clave primaria (`id_problema`) de los registros que violaron las reglas.
4. **Separación y Ruteo:** Usando los identificadores fallidos, el código divide dinámicamente los datos: los registros sanos son promovidos a la capa Silver, mientras que los defectuosos son desviados a una Dead Letter Queue para su revisión, garantizando un ecosistema analítico libre de contaminación.

5.2 Assertions de Calidad Aplicadas al Gobierno

Las reglas implementadas no son arbitrarias, sino que responden directamente a las necesidades del negocio definidas en el catálogo:

- **1. Integridad de Entidades (Unicidad de id_problema):** Exigida por auditoría. Un ticket duplicado en el Data Lake distorsionaría de forma directa los volúmenes y las métricas operativas calculadas posteriormente en Spark SQL.
- **2. Restricción de Ciclo de Vida:** Impide que sistemas de terceros inyecten tickets con estados atípicos o no reconocidos por la empresa (ej. "En Pausa" o "En Análisis"), forzando a que operen bajo el dominio oficial ("Resuelto" o "No Resuelto").
- **3. Completitud de Atributos (fecha_origen Not Null):** Garantiza que todo incidente posea obligatoriamente una marca temporal al ingresar al ecosistema. Esto es un requisito indispensable para poder auditar tiempos de respuesta y Acuerdos de Nivel de Servicio.
- **4. Estandarización Sintáctica:** Fuerza estrictamente el formato internacional ISO 8601 (YYYY-MM-DD). Esta regla evita que la ingesta de formatos locales (ej. DD/MM/YYYY) genere excepciones de casteo al ser consumidos por analistas en herramientas de BI.
- **5. Referencialidad Lógica:** Dado que en la capa Bronze (almacenamiento en Data Lake) no existen Claves Foráneas (FK) físicas, esta regla emula la validación relacional, asegurando que ningún incidente reporte una caída apuntando a un servidor con un identificador negativo o nulo.

5.3 Código Implementado

A continuación, se detalla la función que materializa esta lógica, recibiendo los datos crudos y devolviendo los datasets limpios y de cuarentena:

```
1 def ejecutar_pipeline_calidad(df_entrada, titulo_fase):  
2     # A. Configuración y Carga  
3     context = gx.get_context(mode="ephemeral")
```

```
4 data_source = context.data_sources.add_pandas(name="ds_incidentes")
5 data_asset = data_source.add_dataframe_asset(name="asset_incidentes")
6 batch_def = data_asset.add_batch_definition_whole_dataframe("batch")
7
8 # B. Definicion de Reglas de Negocio
9 suite = context.suites.add(gx.ExpectationSuite(name="suite_silver"))
10 suite.add_expectation(gx.expectations.ExpectColumnValuesToBeUnique(
column="id_problema"))
11 suite.add_expectation(gx.expectations.ExpectColumnValuesToBeInSet(
12     column="estado", value_set=["Resuelto", "No Resuelto"]
13 ))
14 suite.add_expectation(gx.expectations.ExpectColumnValuesToNotBeNull(
column="fecha_origen"))
15 suite.add_expectation(gx.expectations.ExpectColumnValuesToMatchRegex(
16     column="fecha_origen", regex=r"^\d{4}-\d{2}-\d{2}$"
17 ))
18 suite.add_expectation(gx.expectations.ExpectColumnValuesToBeBetween(
19     column="id_fuente", min_value=1
20 ))
21
22 # C. Ejecucion y extraccion de IDs problematicos
23 validation_def = context.validation_definitions.add(
24     gx.ValidationDefinition(name="val_silver", data=batch_def, suite=
suite)
25 )
26 results = validation_def.run(
27     batch_parameters={"dataframe": df_entrada},
28     result_format={
29         "result_format": "COMPLETE",
30         "unexpected_index_column_names": ["id_problema"]
31     }
32 )
33
34 ids_para_cuarentena = set()
35 for res in results.results:
36     if not res.success:
37         unexpected_idx = res.result.get("unexpected_index_list", [])
38         ids_fallidos = [r["id_problema"] for r in unexpected_idx] if
unexpected_idx else []
```

```

39         ids_para_cuarentena.update(ids_fallidos)
40
41     # D. Separacion y Ruteo
42     df_dlq = df_entrada[df_entrada["id_problema"].isin(ids_para_cuarentena
43 )]
44     df_silver = df_entrada[~df_entrada["id_problema"].isin(
45         ids_para_cuarentena)]
46
47     return df_silver, df_dlq

```

Listing 7: Implementación del Data Quality Gate y Ruteo hacia DLQ

5.4 Validación y Análisis de Resultados

Para comprobar la eficacia de esta arquitectura, el proceso de validación se ejecutó en dos fases secuenciales: un escenario de control con datos estructurados y una prueba de estrés con anomalías simuladas.

Fase 1: Escenario de Control (Datos Limpios)

En primera instancia, el motor evaluó el recorte original de 20 registros extraídos directamente del DML relacional (PostgreSQL).

Resultado obtenido:

EJECUTANDO BARRERA DE CALIDAD: Fase PoC Inicial - Dataset Relacional

Tabla 1: Resultado de las reglas de calidad

Regla de Calidad	Estado	Diagnóstico Técnico
Regla 1 (ID Único)	Éxito	Operación limpia sobre la columna.
Regla 2 (Estado válido)	Éxito	Operación limpia sobre la columna.
Regla 3 (Fecha no nula)	Éxito	Operación limpia sobre la columna.
Regla 4 (Formato Fecha ISO)	Éxito	Operación limpia sobre la columna.
Regla 5 (ID Fuente > 0)	Éxito	Operación limpia sobre la columna.

Métricas finales del pipeline

Métrica	Valor
Total registros procesados en Capa Bronze	20
Registros promovidos a Capa Silver	20
Registros rechazados (ruteados a DLQ)	0

Tal como se esperaba, la totalidad del lote fue promovido a la capa Silver sin rechazos. Este escenario de control cumple un propósito: confirmar que el pipeline no introduce falsos positivos sobre datos bien formados. En un entorno de producción real, esta validación previa sería condición necesaria antes de exponer el motor a ingestas externas no controladas, garantizando que la barrera de calidad discrimine con precisión y no penalice registros legítimos.

Fase 2: Prueba de Estrés e Inyección de Anomalías

Para evaluar la resiliencia del pipeline ante ingestas no controladas (típicas en entornos de Big Data), inyectamos intencionalmente 5 registros malformados al lote original: un identificador duplicado, un estado inexistente ("En Análisis"), una fecha nula, una fecha con formato latinoamericano y un ID de fuente negativo.

Resultado obtenido tras la ejecución:

EJECUTANDO BARRERA DE CALIDAD: Fase Estresada - Ingesta con Datos Corruptos

Tabla 2: Resultado de las reglas de calidad

Regla de Calidad	Estado	Diagnóstico Técnico
Regla 1 (ID Único)	Fallo	Valores duplicados en <code>id_problema</code> .
Regla 2 (Estado válido)	Fallo	Valores fuera del dominio permitido.
Regla 3 (Fecha no nula)	Fallo	Atributo obligatorio ausente.
Regla 4 (Formato Fecha ISO)	Fallo	Formato sintáctico inválido.
Regla 5 (ID Fuente > 0)	Fallo	Clave foránea referencial inconsistente.

Métricas finales del pipeline

Métrica	Valor
Total registros procesados en Capa Bronze	25
Registros promovidos a Capa Silver	19
Registros rechazados (ruteados a DLQ)	6

Contenido en la Dead Letter Queue (DLQ)

ID	Descripción	Estado	ID Fuente
1	Datos duplicados en tabla de clientes	Resuelto	2
1	Falla de conexión (ID duplicado)	No Resuelto	4
21	Error desconocido en origen	En Análisis	2
22	Tabla borrada accidentalmente	No Resuelto	6
23	Lentitud extrema en réplica	No Resuelto	8
24	Logs corruptos de auditoría de discos	No Resuelto	-5

La prueba de estrés demostró la robustez del Data Quality Gate. El sistema capturó con precisión quirúrgica todas las violaciones inyectadas. Resulta especialmente destacable el comportamiento ante la violación de unicidad (Regla 1): al detectar que el registro inyectado compartía el `id_problema = 1` con un registro legítimo, el motor envió ambos a la

cuarentena (explicando por qué hay 6 registros rechazados frente a 5 inyectados). Esto garantiza que no se procesen datos ambiguos en etapas posteriores.

Al igual que en las fases de procesamiento distribuido, esta prueba sobre una muestra controlada validó la eficacia del componente técnico propuesto para la arquitectura.

Comprobamos empíricamente que el mecanismo de Dead Letter Queue funciona tal como fue diseñado, logrando separar de forma automatizada los datos limpios de los corruptos, y preservando la trazabilidad de los errores para el equipo de Data Stewards. En síntesis, esta prueba garantiza que, al escalar a la volumetría real corporativa, el gobierno de datos estará protegido por una barrera sólida, asegurando que los reportes ejecutivos operen exclusivamente sobre información certificada.

6 Persistencia Políglota y Almacenamiento Documental

Para culminar el diseño arquitectónico distribuido, se evaluó la incorporación de una estrategia de Persistencia Políglota. Mientras que el núcleo estructural del Catálogo Maestro reside de forma óptima en el motor transaccional normalizado (PostgreSQL), existen entidades operacionales que poseen esquemas dinámicos que cambian constantemente. Forzar estas estructuras polimórficas dentro de una rigidez relacional obligaría a diseñar múltiples tablas puente y costosas operaciones de JOIN para leer un único evento. Para validar la viabilidad de un almacenamiento híbrido, se implementó una prueba enfocada en la gestión de incidentes.

6.1 Documentación de la Infraestructura Elegida y Credenciales

El entorno de validación se instanció en la nube utilizando **MongoDB Atlas** bajo un modelo Database-as-a-Service (DBaaS), aislando la infraestructura analítica del mantenimiento de servidores físicos.

Nota para la evaluación: Con el propósito de garantizar la reproducibilidad completa del pipeline de manera externa, se creó un usuario de base de datos exclusivo para la corrección, configurando el clúster con acceso de red global (0.0.0.0/0) para mitigar bloqueos por IPs dinámicas. No se requiere ingresar a la interfaz web de MongoDB Atlas; la cadena de conexión (`MONGO_URI`) integrada en el código del entorno Google Colab ya posee incrustadas las credenciales automatizadas:

- **Usuario de Conexión:** `alumno_td`
- **Contraseña de Conexión:** `gobiernodedatos`

6.2 Diseño de la Colección Documental

Utilizando un recorte representativo de la tabla `PROBLEMA` (incidentes de calidad de datos) derivado del DML original, se modeló una colección denominada `tickets_itsm`. Cada documento JSON dentro del clúster explota las capacidades nativas del modelo NoSQL documental:

- **Arrays Dinámicos:** El campo `etiquetas` almacena una lista de palabras clave indexables de cardinalidad variable, evitando tablas puente de muchos a muchos.
- **Sub-documentos Embebidos (Desnormalización):** El objeto `fuente_afectada` encapsula los metadatos de infraestructura dentro del propio ticket, acelerando las lecturas.
- **Historial Cronológico Anidado:** El array de objetos `historial_resolucion` crece de forma inmutable, registrando las acciones del equipo técnico y los comentarios operativos en una sola localización física del disco.

6.3 Casos de Uso Relevados y Consultas Implementadas

A continuación, se describen los tres escenarios analíticos ejecutados sobre la muestra para validar la flexibilidad y el valor de negocio de la consulta sobre documentos semi-estructurados:

1. Filtrado Jerárquico sobre Sub-documentos y Arrays

El objetivo de la consulta es poder localizar incidentes críticos basándose en clasificaciones dinámicas de texto combinadas con atributos jerárquicos del servidor afectado.

```
1 query_1 = {  
2     "etiquetas": "urgente",  
3     "fuente_afectada.tipo": "Relacional"  
4 }  
5 # Se aplica una proyeccion para ocultar el ObjectId interno de MongoDB  
6 for doc in coleccion_tickets.find(query_1, {"_id": 0, "id_problema": 1, "  
    descripcion": 1, "etiquetas": 1}):  
7     print(doc)
```

Listing 8: Filtrado por notación de punto y arrays implícitos

Resultado obtenido en consola:

```
1 {'id_problema': 10, 'descripcion': 'Timeout en query de tablero  
    ejecutivo',  
2  'etiquetas': ['timeout', 'performance', 'urgente', 'oracle']}
```

Análisis: La consulta validó la efectividad de la búsqueda analítica en milisegundos sin recurrir a uniones lógicas. MongoDB resolvió de forma nativa la localización del término "urgente" dentro de una lista dinámica, y navegó jerárquicamente hacia el sub-documento mediante sintaxis de punto (*Dot Notation*), aislando exitosamente el ticket buscado.

2. Extracción Analítica mediante Aggregation Framework

El objetivo es generar un reporte plano para auditorías de Recursos Humanos enfocado en aislar cronológicamente cada intervención operativa realizada por el rol "Data Engineer" a través de todo el histórico de tickets.

```
1 pipeline = [  
2     {"$unwind": "$historial_resolucion"},  
3     {"$match": {"historial_resolucion.actor": "Data Engineer"}},  
4     {"$project": {  
5         "_id": 0, "id_problema": 1,  
6         "accion_realizada": "$historial_resolucion.accion",  
7         "fecha_accion": "$historial_resolucion.fecha"  
8     }}  
9 ]  
10 for res in coleccion_tickets.aggregate(pipeline):  
11     print(res)
```

Listing 9: Aggregation Pipeline para aplanamiento de históricos

Resultado obtenido en consola:

```
1 {'id_problema': 13, 'accion_realizada': 'Reinicio de brokers', '  
    fecha_accion': '2023-11-02T09:00:00'}  
2 {'id_problema': 1, 'accion_realizada': 'Cierre de ticket', '  
    fecha_accion': '2022-04-10T16:45:00'}
```

Análisis: El flujo secuencial confirmó la viabilidad del aplanamiento dinámico de datos. La etapa `unwind` descompuso el array interno multiplicando virtualmente el documento matriz, permitiendo filtrar la auditoría y moldear la proyección final. Esto demuestra que es posible almacenar datos anidados complejos orientados a eventos y consumirlos en formato tabular lineal cuando un analista corporativo lo requiera.

3. Consolidación de KPIs Gerenciales mediante Agrupación

El objetivo es generar estadísticas de rendimiento global calculando el volumen de incidentes ocurridos según la naturaleza técnica de la infraestructura y cuantificando las penalizaciones asociadas al incumplimiento de los Acuerdos de Nivel de Servicio (SLA), es decir, aquellos casos en los que los tiempos de resolución o atención excedieron los límites comprometidos contractualmente.

```
1 pipeline_kpi = [  
2     {"$group": {
```

```
3     "_id": "$fuente_afectada.tipo",
4     "volumen_incidentes": {"$sum": 1},
5     "tickets_sla_vencido": {
6         "$sum": {"$cond": [{"$eq": ["$sla_vencido", True]}, 1, 0]}
7     }
8 },
9 {"$project": {
10     "tipo_infraestructura": "$_id", "_id": 0,
11     "volumen_incidentes": 1, "tickets_sla_vencido": 1
12 }}
13 ]
14 for res in coleccion_tickets.aggregate(pipeline_kpi):
15     print(res)
```

Listing 10: Aggregation Pipeline con agrupación analítica y condicionales

Resultado obtenido en consola:

```
1 {'volumen_incidentes': 1, 'tickets_sla_vencido': 0, '
   tipo_infraestructura': 'No Relacional'}
2 {'volumen_incidentes': 2, 'tickets_sla_vencido': 1, '
   tipo_infraestructura': 'Relacional'}
```

Análisis: Esta consulta demostró la potencia analítica del motor documental. Combinando la agrupación distributiva de `group` con acumuladores aritméticos lógicos (`cond`), se consiguió emular de manera nativa el comportamiento de las sentencias avanzadas de SQL (`GROUP BY` y `SUM(CASE WHEN)`). El *insight* arrojado extrajo con precisión que la muestra relacional concentra el mayor riesgo latente y la única penalización de SLA activa. Es importante destacar que, dado el volumen acotado de la muestra (3 documentos), esta conclusión debe interpretarse como una validación técnica del pipeline y no como un hallazgo estadísticamente representativo. En un entorno de producción real con una colección de mayor escala, este mismo agregador permitiría identificar patrones de riesgo estructurales por tipo de infraestructura con alto grado de confianza.

Al igual que en los motores previos, la experimentación sobre el recorte del catálogo validó

conceptualmente el rol de la persistencia políglota dentro de la arquitectura. La prueba permitió demostrar que forzar entidades altamente dinámicas dentro de PostgreSQL introduce una complejidad innecesaria en la base de datos de origen.

A través del análisis de estos JSONs anidados, comprobamos empíricamente que la separación de cargas es la estrategia óptima: mientras el glosario estático del catálogo se beneficia de la consistencia transaccional ACID relacional, los componentes vivos de auditoría y soporte logran un rendimiento superior al ser desnormalizados bajo un esquema documental. En síntesis, se determinó la factibilidad técnica del entorno híbrido en la nube, garantizando un ecosistema ágil, escalable y con múltiples interfaces adaptadas a los distintos consumidores de gobierno de la empresa.

7 Conclusión

El presente trabajo práctico demostró de manera empírica la evolución de un modelo de Gobierno de Datos tradicional hacia una arquitectura analítica moderna, escalable y automatizada. Partiendo del diseño transaccional y estático definido en la primera etapa (PostgreSQL), logramos identificar sus limitaciones físicas frente a las demandas de un entorno de Big Data y diseñamos un ecosistema que se adapta a las demandas exigidas.

A través del despliegue de pruebas sobre un entorno simulado, validamos exitosamente los tres pilares fundamentales de nuestra propuesta técnica:

1. **Defensa y Calidad Automatizada:** La implementación de Great Expectations actuó como un Data Quality Gate implacable. Comprobamos que es posible aislar la "basura" operativa mediante una Dead Letter Queue antes de que contamine las capas de consumo, garantizando que la gerencia y los modelos predictivos operen exclusivamente sobre datos certificados.
2. **Gobernanza Analítica sin Fricción Operativa:** Mediante los paradigmas de MapRe-

duce y Spark SQL, demostramos que la organización puede ejecutar auditorías de seguridad perimetral, trazar el linaje de sus activos (Compliance) y calcular el impacto de su deuda técnica de manera distribuida, sin penalizar ni bloquear las bases de datos operacionales.

3. **Flexibilidad mediante Persistencia Políglota:** Al desviar el flujo de tickets de incidentes hacia MongoDB Atlas, resolvimos el cuello de botella que suponía modelar entidades dinámicas en tablas relacionales rígidas. Validamos que una arquitectura híbrida proporciona la agilidad necesaria para soportar historiales orientados a eventos, habilitando analítica compleja directamente sobre documentos JSON.

En síntesis, este diseño arquitectónico transforma el Gobierno de Datos: deja de ser un proceso burocrático, manual y reactivo, para convertirse en un producto automatizado y resiliente. El resultado de esta simulación garantiza que, al escalar hacia los volúmenes reales de la corporación, la organización contará con la visibilidad, trazabilidad y seguridad necesarias para convertir su ecosistema de datos en una verdadera ventaja competitiva.